Page No.:
Date: / /

Lecture - 14

Binary Search

* Find smallest absent element :

Example :

0	1	2	3	4	5	6
0	1	2	3	4	6	7

↑

(5) missing

```
for (int i = 0; i <= n; i++)
    if (arr[i] != i)                    O(n)
        return i;
return n;
```

By Binary search :

0	1	2	3	4	5
0	1	2	3	5	8
low					high

$$mid = \frac{0+5}{2} = 2$$

arr[mid] = mid (true)

→ index 2 tak sahi hai ab mid ke
बाद check करेंगे

start = mid + 1.

Teacher's Signature.....

3	4	5
3	5	8

low high

$mid = \frac{3+5}{2} = 4$

arr[mid] == mid (false)

mismatched found

ans = arr[mid] = 5

↳ check in left

high = mid - 1.

3
3

low
high

$mid = \frac{3+3}{2} = 3$

arr[mid] = mid (true)

Final answer

↳ index = 4

element = 5

Code:

```
int start = 0, end = N-1, mid, index;
while (start <= end) {
    mid = end + (start - end)/2;
    if (mid < arr[mid]) {
        index = mid;
        end = mid - 1;
    }
```

Teacher's Signature.....

```
        else {
            index = mid + 1;
            start = mid + 1;
        }
    }

    return mid;
}
```

* Rotate Array :

Arr : Sorted in ascending, All are distinct Element.

Size : N

Rotate D times

D = 0	1	2	3	(4)	} repeat this
D = 1	4	1	2	(3)	
D = 2	3	4	1	(2)	
D = 3	2	3	4	(1)	
D = 4	1	2	3	4	

On the the series is repeat after N Element.

Teacher's Signature.....

Page No.:
Date: / /

* search in a rotated array :

Sorted and rotated array
size = N distinct element

Find index of given element

N = 9

A[] = { 5, 6, 7, 8, 9, 10, 1, 2, 3 }

key = 10

l = 0 h = 8

output = 5

If Not present : return -1,

0	1	2	3	4	5	6	7	8
5	6	7	8	9	10	1	2	3

↓
pivot

key = 10

```
if (arr[mid] == key)
    return mid;
```

$mid = \frac{0+8}{2} = 4$

arr[4] = 9

(9 < 10)

Teacher's Signature.....

{ arr[mid] < target }

↳ two Condition

1) A[l] <= key || key < A[mid]

↳ end = mid - 1

2) Condition 2

↳ start = mid + 1;

{ arr[mid] > target }

↳ if (arr[start] < target)

↳ end = mid - 1;

↳ if (arr[start] > target)

↳ start = mid + 1;

code :

```
int search ( int A[], int l, int h, int key) {
    int index = -1;
    while (l <= h) {
        int mid = h + (l - h) / 2;

    if ( A[mid] == key){
        return mid;
    }
    else if ( A[mid] > A[l]) {
        if ( A[mid] < key || key < A[l]) {
            l = mid + 1;
        }
```

```
        else {
            h = mid - 1;
        }
    }
    else {
        if (A[l] <= key || key < A[mid]) {
            h = mid - 1;
        }
        else {
            l = mid + 1;
        }
    }
    return index;
  }
};
```

* Allocate Minimum Number of pages :

Given : N Books
Each have Ai pages
M Students
need to allocated Contiguous Books.
Each student get atleast one Book.

0	1	2	3
10	15	10	20

M = 2

Teacher's Signature.....

	A	B		Diff
10	10	15, 10, 20	45	35
25	10, 15	10, 20	30	(5) ✓
35	10, 15, 10	20	20	15

In this we can use binary search

Example:

N = 4

A[] = {12, 34, 67, 90}

student M = 2

{12}	{34, 67, 90}	Max : 191
{12, 34}	{67, 90}	Max : 157
{12, 34, 67}	{90}	Max = 113

Take Min ↓ Answer.

Answer nahi mil rha hai to
↳ Right side

Answer mil rha hai to
↳ Left side.

Let Explain With an Example:

Teacher's Signature.....

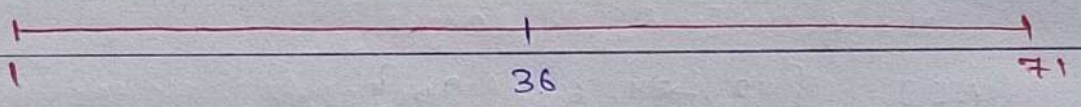
Example:

0	1	2	3
10	20	10	31

Start = 1

End = sum of No. of pages

end = 71



$$mid = \frac{start + end}{2} = 36$$

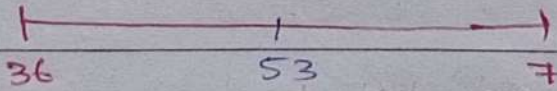
↳ Ek student

S1 = 10 + 20 + ~~10~~ ko Max 36 page.

S2 = 10 + ~~31~~ Not possible

S3 = 31

go to Right side.



$$mid = \frac{36 + 71}{2} = \frac{107}{2} = 53$$

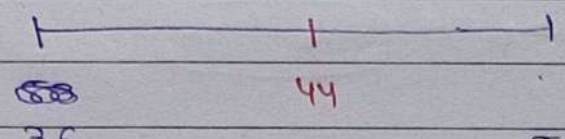
possible { S1 = 10 + 20 + 10 + ~~31~~

S2 = 31

Go to left.

Teacher's Signature.....

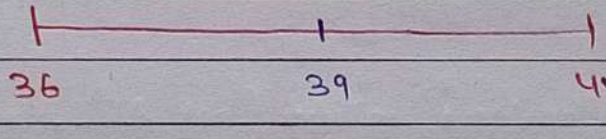
S3 पे Answer मिल रहा है लेकिन ये possibility hai Ki usse kam pe bhi answer Mil Jaye.



$$Mid = \frac{36+53}{2} = 44$$

possible { S1 = 10 + 20 + 10 + ~~1~~
S2 = 31

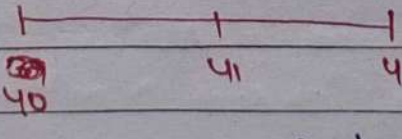
Left



$$Mid = \frac{36+43}{2} = 39$$

Not possible { S1 = 10 + 20 + ~~10~~
S2 = 10 + ~~21~~
S3 = 31

Go to Right

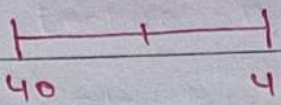


$$Mid = \frac{39+43}{2} = \frac{82}{2} = 41$$

possible { S1 = 10 + 20 + 10 + ~~21~~
S2 = 31

Go to left

Teacher's Signature.....

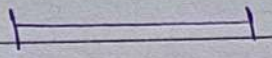


S1 = 10+20+10 (Mid = 40)

S2 = 31

possible left

Condition fails:



{ Answer : 40 }

Code :

```
Public Static boolean isTrue (int A[], int N, int M,
                              int mid) {
    int student = 1;
    int sum = 0;

    for( int i = 0; i<N; i++) {
        sum = sum + A[i];
        if (sum > mid) {
            Student++;
            Sum = A[i];
            if ( student > M) {
                return false;
            }
        }
    }
    return true;
}
```

Teacher's Signature.....

```
public static int findPages(int []A, int N, int M){

        IF (N < M) return -1;
        int sum = 0;
        int start = A[0];
        int ans = Integer.MAX_VALUE;
       for(int i=0; i<N; i++){
              Sum += A[i];

            if( A[i] > start){
                  start = A[i];
            }
    }
    int end = sum;
    While ( start <= end){
        int mid = start + (end - start)/2;

      if(isTrue( A, N, M, mid){
              ans = Math.min (ans, mid);
              end = mid - 1;
       }
       else {
              start = mid + 1;
       }
    }
    return ans;
  }
};
```